📑 Basic Information:

Experiment No: 12

Date:

Title: Implement a Reservation system which allows the persons to book seats. Define reserve method initially with 100 seats. Now create one or more person threads to book seats. Atany time it should allow only one person thread to access the reserve method.

🎯 Objective /Aim:

To demonstrate Java multithreading and synchronization by safely reserving shared seats, preventing multiple threads from reserving more seats than are available.

🛠️ System Requirements:

Hardware:

-> Standard PC/Laptop

-> Intel Core i3/i5 processor or equivalent

-> Minimum 4 GB RAM

-> Minimum 500 MB free

Software:

->Operating System: Windows / Linux / macOS

-> Java Development Kit (JDK 17 or JDK 21)

-> Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code

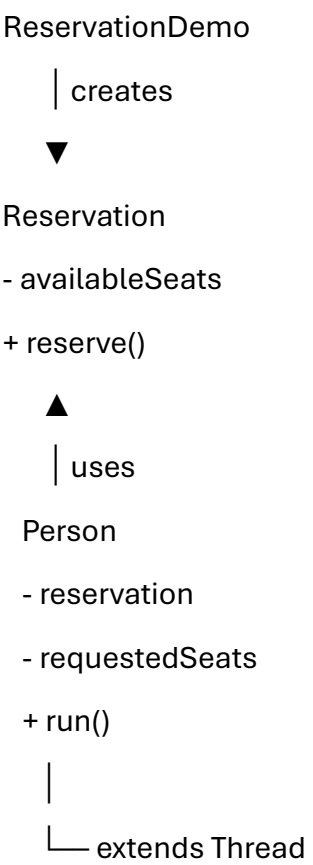
-> Command Prompt / Terminal

Algorithm/Logic Description:

1. Start the program.
2. Create a Reservation object with 10 available seats.
3. Create three Person threads that share the same reservation object:
    - Person-1 requests 5 seats.
    - Person-2 requests 2 seats.
    - Person-3 requests 4 seats.
4. Start Person-1 and wait for it to finish using join().

5. Start Person-2 and wait for it to finish using join().
6. Start Person-3.
7. In each thread, call the synchronized reserve() method:
    - Display the available and requested seats.
    - If enough seats are available, reserve them and update the count.
    - Otherwise, display that the seats are unavailable.
8. Stop the program.

UML Class Diagram:



Source Code:

```
// Manages the shared seat count and reservation process
class Reservation {
   int availableSeats = 10;
  // Allows only one thread at a time to check and reserve seats
   synchronized void reserve(String name, int requestedSeats) {
      System.out.println(name + " entered.");
```

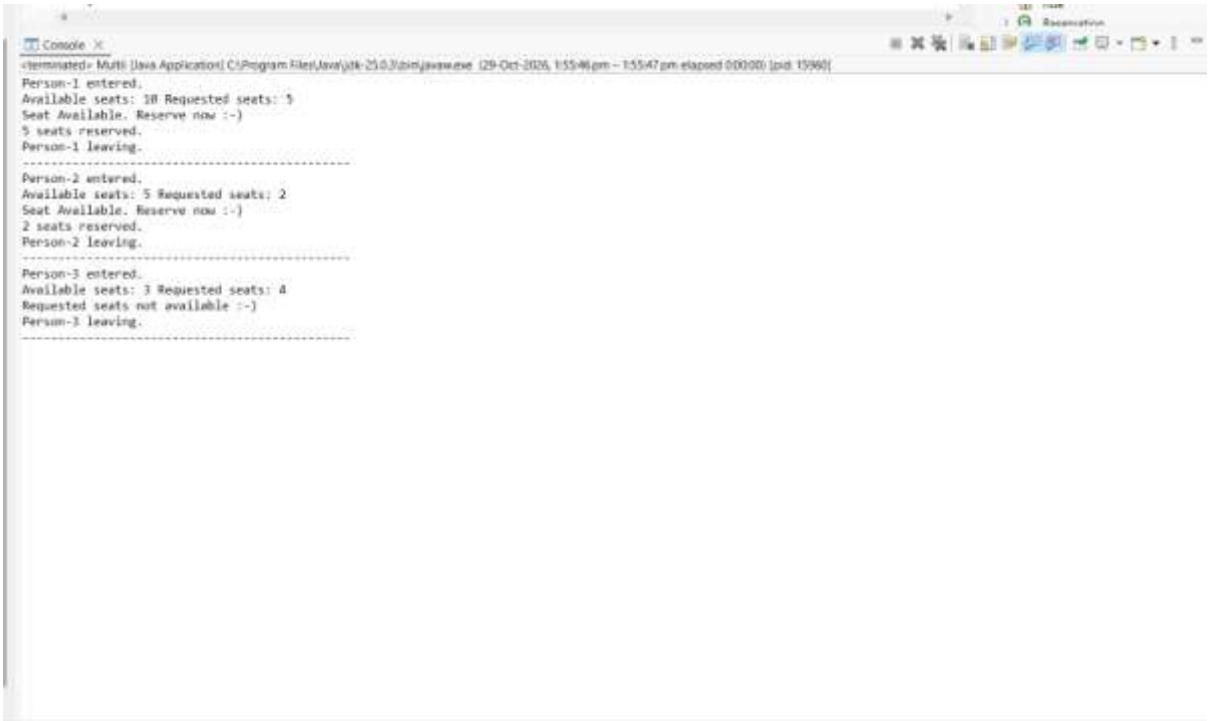
```
System.out.println("Available seats: " + availableSeats+ " Requested seats: " +
requestedSeats);
      // Check whether enough seats are available
      if (requestedSeats <= availableSeats) {
         System.out.println("Seat Available. Reserve now :-)");
         availableSeats = availableSeats - requestedSeats;
         System.out.println(requestedSeats + " seats reserved.");
      } else {
         System.out.println("Requested seats not available :-)");
      }
        System.out.println(name + " leaving.");
       System.out.println("-----------------------------------------------");
   }
}
// Each Person runs as a separate thread
class Person extends Thread {
   Reservation reservation;
   int requestedSeats;
  // Set the shared reservation, thread name, and number of seats requested
   Person(Reservation reservation, String name, int requestedSeats) {
      super(name);
      this.reservation = reservation;
      this.requestedSeats = requestedSeats;
   }
   // Thread execution calls the synchronized reservation method
   public void run() {
      reservation.reserve(getName(), requestedSeats);
   }
}
// Main class that creates and starts the person threads
```

```
public class ReservationDemo {
   public static void main(String[] args) {
      // All threads share this reservation object
      Reservation r = new Reservation();
      // Create three people with their seat requests
      Person p1 = new Person(r, "Person-1", 5);
      Person p2 = new Person(r, "Person-2", 2);
      Person p3 = new Person(r, "Person-3", 4);
      // Start Person-1 and wait until its reservation attempt finishes
      p1.start();
      try {
         p1.join();
      } catch (InterruptedException e) {
         // Restore the interrupted status and stop the main thread
         Thread.currentThread().interrupt();
         return;
      }
      // Start Person-2 and wait until its reservation attempt finishes
      p2.start();
      try {
         p2.join();
      } catch (InterruptedException e) {
         // Restore the interrupted status and stop the main thread
         Thread.currentThread().interrupt();
         return;
      }
      // Start Person-3; main does not wait for this thread to finish
      p3.start();
   }}
```

Relationship:

- Person extends the Java Thread class.
- Each Person has a reference to a shared Reservation object and calls its synchronized reserve() method.
- ReservationDemo creates one Reservation and three Person threads.

Output Screenshot:



Result:

The program demonstrates Java multithreading and synchronization. The synchronized reservation method protects the shared seat count, ensuring reservations are checked and updated safely. With the given requests, the first two people reserve seats, while the last request cannot be fulfilled because too few seats remain.